

EXP20

```
1
2  people = {
3      "Sai": "15-06-2005",
4      "Rahul": "20-08-2004",
5      "Priya": "10-12-2005",
6      "Kiran": "25-03-2004"
7  }
8  name = input("Enter person name: ").capitalize()
9  if name in people:
10     dob = people[name]
11     print(name, "was born on", dob)
12 else:
13     print("Person not found")
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> & "C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab\EXP20"
Enter person name: kiran
Kiran was born on 25-03-2004
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> |
```

EXP21 > ...

```
1  print("Pattern Recognition System")
2  sides = int(input("Enter number of sides: "))
3  area = float(input("Enter area of the shape: "))
4  if sides == 0 and area > 0:
5      pattern = "Circle"
6  elif sides == 3:
7      pattern = "Triangle"
8  elif sides == 4:
9      pattern = "Square or Rectangle"
10 elif sides == 5:
11     pattern = "Pentagon"
12 elif sides == 6:
13     pattern = "Hexagon"
14 else:
15     pattern = "Unknown Pattern"
16
17 print("Recognized Pattern:", pattern)
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> & "C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab\EXP21"
```

Pattern Recognition System

Enter number of sides: 3

Enter area of the shape: 6

Recognized Pattern: Triangle

```
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> █
```

```

2  state = {
3      "monkey": "door",
4      "box": "window",
5      "banana": "middle",
6      "on_box": False,
7      "has_banana": False
8  }
9  print("Initial State:", state)
10 goal = True
11 state["monkey"] = state["box"]
12 print("\nAction 1: Monkey moves to the box")
13 state["box"] = state["banana"]
14 state["monkey"] = state["banana"]
15 print("Action 2: Monkey pushes the box under the banana")
16 state["on_box"] = True
17 print("Action 3: Monkey climbs onto the box")
18 if state["on_box"] and state["monkey"] == state["banana"]:
19     state["has_banana"] = True
20     print("Action 4: Monkey takes the banana")
21 if state["has_banana"] == goal:
22     print("\nGoal Achieved!")
23     print("Monkey has the banana.")
24 else:
25     print("\nGoal Not Achieved")

```

Problems Output Debug Console **Terminal** Ports

Action 1: Monkey moves to the box
 Action 2: Monkey pushes the box under the banana
 Action 3: Monkey climbs onto the box
 Action 4: Monkey takes the banana

Goal Achieved!
 Monkey has the banana.

PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> █

```
1 # Forward Chaining - Computer Fault Diagnosis
2
3 print("Computer Fault Diagnosis")
4 computer_on = input("Is the computer ON? (yes/no): ").lower()
5 screen_blank = input("Is the screen blank? (yes/no): ").lower()
6 fan_running = input("Is the fan running? (yes/no): ").lower()
7 if computer_on == "no":
8     print("\nConclusion: Power Problem")
9 elif computer_on == "yes" and screen_blank == "yes":
10    print("\nConclusion: Display Problem")
11 elif computer_on == "yes" and fan_running == "no":
12    print("\nConclusion: Overheating Problem")
13 else:
14    print("\nConclusion: No fault detected")
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> & "C:\Users\VENKAT SAI.J\AppData\Local\Programs\Python\Python310\python.exe" "C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab\EXP23.py"
```

```
Computer Fault Diagnosis
```

```
Is the computer ON? (yes/no): yes
```

```
Is the screen blank? (yes/no): no
```

```
Is the fan running? (yes/no): yes
```

```
Conclusion: No fault detected
```

```
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> █
```

```
1  # Backward Chaining - Medical Diagnosis
2
3  print("Medical Diagnosis")
4  fever = input("Do you have fever? (yes/no): ").lower()
5  cough = input("Do you have cough? (yes/no): ").lower()
6  body_pain = input("Do you have body pain? (yes/no): ").lower()
7  sneezing = input("Do you have sneezing? (yes/no): ").lower()
8  print("\nChecking possible diseases...")
9  if fever == "yes" and cough == "yes" and body_pain == "yes":
10     print("Conclusion: Possible Flu")
11  elif cough == "yes" and sneezing == "yes":
12     print("Conclusion: Possible Common Cold")
13  else:
14     print("Conclusion: No matching disease found")
```

Problems Output Debug Console **Terminal** Ports

Medical Diagnosis

Do you have fever? (yes/no): yes

Do you have cough? (yes/no): yes

Do you have body pain? (yes/no): yes

Do you have sneezing? (yes/no): no

Checking possible diseases...

Conclusion: Possible Flu

PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> █

EXP25.pl

```
1
2 % Best First Search
3 edge(a, b).
4 edge(a, c).
5 edge(b, d).
6 edge(b, e).
7 edge(c, f).
8 edge(c, g).
9 % Heuristic values
10 h(a, 6).
11 h(b, 4).
12 h(c, 3).
13 h(d, 5).
14 h(e, 2).
15 h(f, 1).
16 h(g, 0).
17 best_first(Start, Goal) :-
18     search([Start], Goal).
19 search([Goal|_], Goal) :-
20     write('Goal reached: '),
21     write(Goal), nl.
22 search([Current|Rest], Goal) :-
23     findall(H-Node,
24         (edge(Current, Node),
25          h(Node, H),
26          \+ member(Node, [Current|Rest])),
27         Children),
28     sort(Children, Sorted),
29     Sorted = [_-Next|_],
30     write('Visiting: '),
31     write(Next), nl,
32     search([Next, Current|Rest], Goal).
```

Problems Output Debug Console Terminal Ports

PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab>

```

UCS.py > ucs
1 import heapq
2 graph={
3     'A':[( 'B',2),('C',4)],
4     'B':[( 'D',3),('E',1)],
5     'C':[( 'F',5)],
6     'D':[],
7     'E':[( 'F',2)],
8     'F':[]
9 }
10 def ucs(start,goal):
11     queue=[(0,start)]
12     while queue:
13         cost,node=heapq.heappop(queue)
14         if node==goal:
15             print("minimum cost=",cost)
16             return
17         for next_node,weight in graph[node]:
18             heapq.heappush(queue,(cost+weight,next_node))
19 start=input("enter start node:")
20 goal=input("enter goal node:")
21 ucs(start,goal)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs> & "C:\Users\VENKAT SAI.J\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/VENKAT SAI.J/OneDrive/Desktop/cv/bfs/UCS.py"

```

```

enter start node:B

```

```

enter goal node:F

```

```

minimum cost= 3

```

```

PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs>

```

```

fwd-i-search: _

```

+ v ... | {} x

powershell

Python

```

import heapq
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [('G', 2)],
    'E': [('G', 4)],
    'F': [('G', 1)],
    'G': []
}
heuristic = {
    'A': 6,
    'B': 4,
    'C': 4,
    'D': 2,
    'E': 2,
    'F': 1,
    'G': 0
}
def a_star(start, goal):
    pq = []
    heapq.heappush(pq, (heuristic[start], 0, start, [start]))
    visited = {}
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g
        if node in visited and visited[node] <= g:
            continue
        visited[node] = g
        for neighbor, cost in graph[node]:
            new_g = g + cost
            new_f = new_g + heuristic[neighbor]
            heapq.heappush(pq, (new_f, new_g, neighbor, path + [neighbor]))
    return None, float("inf")
start = input("Enter Start Node: ").upper()
goal = input("Enter Goal Node: ").upper()

```


Step 0:

Jug A: [] 0/4 L

Jug B: [] 0/3 L

Step 1:

Jug A: [] 0/4 L

Jug B: [■ ■ ■] 3/3 L

Step 2:

Jug A: [■ ■ ■] 3/4 L

Jug B: [] 0/3 L

Step 3:

Jug A: [■ ■ ■] 3/4 L

Jug B: [■ ■ ■] 3/3 L

Step 4:

Jug A: [■ ■ ■ ■] 4/4 L

Jug B: [■ ■] 2/3 L

Goal Reached!

dfs.py > ...

```
1 g={'A':['B','C'],
2   'B':['D'],
3   'C':['E'],
4   'D':[],
5   'E':[]}
6 v=set()
7 s=['A']
8 print("DFS:")
9 while s:
10     n=s.pop()
11     if n not in v:
12         print(n,end=" ")
13         v.add(n)
14         s.extend(reversed(g[n]))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs> & "c:\Users\VENKAT SAI.J\AppData\Local\Programs\Python\Python38-32\python.exe" "c:/Users/VENKAT SAI.J/OneDrive/Desktop/cv/bfs/dfs.py"
```

DFS:

A B D C E

```
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs> █
```

The image shows a Visual Studio Code editor window with a Python file named `bfs.py` open. The file contains a Breadth-First Search (BFS) algorithm implementation. The code is as follows:

```
1 from collections import deque
2 g={'A':['B','C'],
3   'B':['D'],
4   'C':['E'],
5   'D':[],
6   'E':[]}
7 v=set()
8 q=deque(['A'])
9 print("BFS:")
10 while q:
11     n=q.popleft()
12     if n not in v:
13         print(n,end=" ")
14         v.add(n)
15         q.extend(g[n])
```

The terminal at the bottom shows the command to run the script and its output:

```
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs> "c:\Users\VENKAT SAI.J\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/VENKAT SAI.J/OneDrive/Desktop/cv/bfs/bfs.py"
BFS:
A B C D E
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs>
```

On the right side of the editor, there is a 'CHAT' panel with a message: 'Build with Agent AI responses may be in... Generate Agent Instruc... onboard AI onto your c...'.

```

from queue import PriorityQueue
graph = {
    'A': [('B', 4), ('C', 3)],
    'B': [('D', 5), ('E', 12)],
    'C': [('F', 7)],
    'D': [],
    'E': [('G', 2)],
    'F': [('G', 1)],
    'G': []
}
heuristic = {
    'A': 10,
    'B': 8,
    'C': 5,
    'D': 7,
    'E': 2,
    'F': 1,
    'G': 0
}

def best_first_search(start, goal):
    pq = PriorityQueue()
    pq.put((heuristic[start], start))
    visited = set()
    while not pq.empty():
        h, node = pq.get()
        if node in visited:
            continue
        print(node, end=" ")
        visited.add(node)
        if node == goal:
            print("\nGoal reached!")
            return
        for neighbor, cost in graph[node]:
            if neighbor not in visited:
                pq.put((heuristic[neighbor], neighbor))
best_first_search('A', 'F')

```

```

exp5 > ...
1  import heapq
2  graph = {
3      'A': [('B', 1), ('C', 4)],
4      'B': [('D', 2), ('E', 5)],
5      'C': [('F', 3)],
6      'D': [('G', 2)],
7      'E': [('G', 1)],
8      'F': [('G', 2)],
9      'G': []
10 }
11 heuristic = {
12     'A': 7,
13     'B': 6,
14     'C': 4,
15     'D': 2,
16     'E': 1,
17     'F': 2,
18     'G': 0
19 }
20 def greedy_best_first_search(start, goal):
21     queue = [(heuristic[start], start, [start])]
22     visited = []
23     while queue:
24         h, node, path = heapq.heappop(queue)
25         if node == goal:
26             return path
27         if node not in visited:
28             visited.append(node)
29             for neighbor, cost in graph[node]:
30                 if neighbor not in visited:
31                     heapq.heappush(queue, (heuristic[neighbor], neighbor, path + [neighbor]))
32     return None
33 start = input("Enter Start Node: ").upper()
34 goal = input("Enter Goal Node: ").upper()
35 path = greedy_best_first_search(start, goal)

```



```

waterjubprob > print_jugs
1  from collections import deque
2  def print_jugs(a, b, cap1, cap2):
3      print(f"Jug A: [{a} ' ' * a]{' ' * (cap1-a)}] {a}/{cap1} L")
4
5      print(f"Jug B: [{b} ' ' * b]{' ' * (cap2-b)}] {b}/{cap2} L")
6      print("-" * 30)
7  def water_jug(cap1, cap2, goal):
8      visited = set()
9      queue = deque([(0, 0), []])
10     while queue:
11         (a, b), path = queue.popleft()
12         if (a, b) in visited:
13             continue
14         visited.add((a, b))
15         path = path + [(a, b)]
16         if a == goal or b == goal:
17             print("\nSequence of States:\n")
18             step = 0
19             for x, y in path:
20                 print(f"Step {step}:")
21                 print_jugs(x, y, cap1, cap2)
22                 step += 1
23             print("Goal Reached!")
24             return
25         next_states = [
26             (cap1, b),
27             (a, cap2),
28             (0, b),
29             (a, 0),
30             (a - min(a, cap2-b), b + min(a, cap2-b)),
31             (a + min(b, cap1-a), b - min(b, cap1-a))
32         ]
33         for state in next_states:
34             if state not in visited:
35                 queue.append((state, path))
36 water_jug(4, 3, 2)

```

exp7

```
1 def minimax(depth,nodeIndex,maximizingPlayer,values,maxDepth):
2     if depth==maxDepth:
3         return values[nodeIndex]
4     if maximizingPlayer:
5         return max(
6             minimax(depth+1,nodeIndex*2,False,values,maxDepth),
7             minimax(depth+1,nodeIndex*2+1,False,values,maxDepth)
8         )
9     else:
10        return min(
11            minimax(depth+1,nodeIndex*2,True,values,maxDepth),
12            minimax(depth+1,nodeIndex*2+1,True,values,maxDepth)
13        )
14    maxDepth=int(input("enter the depth of the game tree:"))
15    leafNodes=2**maxDepth
16    print(f"enter {leafNodes} leaf node values:")
17    values=[]
18    for i in range(leafNodes):
19        value=int(input(f"leaf{i+1}:"))
20        values.append(value)
21    bestValue=minimax(0,0,True,values,maxDepth)
22    print("\nleaf Node Values:",values)
23    print("optimal value:",bestValue)
24
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

enter the depth of the game tree:2

enter 4 leaf node values:

leaf1:3

leaf2:5

leaf3:2

leaf4:9

leaf Node Values: [3, 5, 2, 9]

optimal value: 3

PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs>

```
exp8 > ...
2 def alphabeta(depth, index, isMax, values, alpha, beta, maxDepth):
3     if depth == maxDepth:
4         return values[index]
5     if isMax:
6         best = -math.inf
7         for i in range(2):
8             value = alphabeta(depth + 1, index * 2 + i, False,
9                               values, alpha, beta, maxDepth)
10            best = max(best, value)
11            alpha = max(alpha, best)
12            if alpha >= beta:
13                break
14        return best
15    else:
16        best = math.inf
17        for i in range(2):
18            value = alphabeta(depth + 1, index * 2 + i, True,
19                              values, alpha, beta, maxDepth)
20            best = min(best, value)
21            beta = min(beta, best)
22            if alpha >= beta:
23                break
24        return best
25 depth = int(input("Enter depth of tree: "))
26 n = 2 ** depth
27 print("Enter", n, "leaf node values:")
28 values = []
29 for i in range(n):
30     values.append(int(input()))
31 result = alphabeta(0, 0, True, values, -math.inf, math.inf, depth)
32 print("Optimal Value =", result)
```

```
exp10 > ...
1 def tower_of_hanoi(n, source, auxiliary, destination):
2     if n == 1:
3         print(f"Move Disk 1 from {source} to {destination}")
4         return
5
6     tower_of_hanoi(n - 1, source, destination, auxiliary)
7     print(f"Move Disk {n} from {source} to {destination}")
8     tower_of_hanoi(n - 1, auxiliary, source, destination)
9
10 # Input
11 n = int(input("Enter the number of disks: "))
12
13 # Function Call
14 tower_of_hanoi(n, 'A', 'B', 'C')
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs> & "C:\Users\VENKAT SAI.J\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/VENKAT SAI.J/OneDrive/Desktop/cv/bfs/exp10"
```

```
Enter the number of disks: 3
```

```
Move Disk 1 from A to C
```

```
Move Disk 2 from A to B
```

```
Move Disk 1 from C to B
```

```
Move Disk 3 from A to C
```

```
Move Disk 1 from B to A
```

```
Move Disk 2 from B to C
```

```
Move Disk 1 from A to C
```

```
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs> |
```

Ln 14, Col 33 S

exp11 > ...

```
1  from itertools import permutations
2
3  letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')
4
5  for p in permutations(range(10), 8):
6      d = dict(zip(letters, p))
7
8      # Leading letters cannot be zero
9      if d['S'] == 0 or d['M'] == 0:
10         continue
11
12     send = 1000*d['S'] + 100*d['E'] + 10*d['N'] + d['D']
13     more = 1000*d['M'] + 100*d['O'] + 10*d['R'] + d['E']
14     money = 10000*d['M'] + 1000*d['O'] + 100*d['N'] + 10*d['E'] + d['Y']
15
16     if send + more == money:
17         print("Solution Found:")
18         print("SEND =", send)
19         print("MORE =", more)
20         print("MONEY =", money)
21         print("\nDigit Assignments:")
22         for key, value in d.items():
23             print(f"{key} = {value}")
24         break
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2

PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs> |

```

from collections import deque
def print_jugs(a, b, cap1, cap2):
    print(f"Jug A: [{a} ' * a}{' ' * (cap1-a)}] {a}/{cap1} L")
    print(f"Jug B: [{b} ' * b}{' ' * (cap2-b)}] {b}/{cap2} L")
    print("-" * 30)
def water_jug(cap1, cap2, goal):
    visited = set()
    queue = deque([(0, 0), []])
    while queue:
        (a, b), path = queue.popleft()
        if (a, b) in visited:
            continue
        visited.add((a, b))
        path = path + [(a, b)]
        if a == goal or b == goal:
            print("\nSequence of States:\n")
            step = 0
            for x, y in path:
                print(f"Step {step}:")
                print_jugs(x, y, cap1, cap2)
                step += 1
            print("Goal Reached!")
            return
        next_states = [
            (cap1, b),
            (a, cap2),
            (0, b),
            (a, 0),
            (a - min(a, cap2-b), b + min(a, cap2-b)),
            (a + min(b, cap1-a), b - min(b, cap1-a))
        ]
        for state in next_states:
            if state not in visited:
                queue.append((state, path))
water_jug(4, 3, 2)

```



```
exp13 > ...
1  from sklearn.neural_network import MLPClassifier
2  X = [
3      [25,120],
4      [30,130],
5      [45,150],
6      [50,160],
7      [35,140],
8      [60,180]
9  ]
10 y = [0,0,1,1,0,1]
11 model = MLPClassifier(hidden_layer_sizes=(5,), max_iter=1000)
12 model.fit(X, y)
13 age = int(input("Enter Age: "))
14 bp = int(input("Enter Blood Pressure: "))
15 result = model.predict([[age, bp]])
16 if result[0] == 1:
17     print("Disease Detected")
18 else:
19     print("No Disease")
```

Problems Output Debug Console Terminal Ports

```
Python\Python314\python.exe" "c:/Users/VENKAT SAI.J/OneDrive/Desktop/cv/bfs/exp13"
Enter Age: 25
Enter Blood Pressure: 170
Disease Detected
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs> |
```

exp12 > ...

```
1  from sklearn.tree import DecisionTreeClassifier
2  X = [
3      [25, 30000],
4      [35, 50000],
5      [45, 60000],
6      [23, 25000],
7      [40, 70000],
8      [30, 40000]
9  ]
10 y = [0, 1, 1, 0, 1, 1]
11 model = DecisionTreeClassifier()
12 model.fit(X, y)
13 age = int(input("Enter Age: "))
14 income = int(input("Enter Annual Income: "))
15 result = model.predict([[age, income]])
16 if result[0] == 1:
17     print("Loan Approved")
18 else:
19     print("Loan Rejected")
```

Problems Output Debug Console Terminal Ports

Python\Python314\python.exe" "c:/Users/VENKAT SAI.J/OneDrive/Desktop/cv/bfs/exp12"

Enter Age: 25

Enter Annual Income: 30000

Loan Rejected

PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs> |

exp15 > [🔍] father

```
1  father = input("Enter Father's Name: ")
2  mother = input("Enter Mother's Name: ")
3  child1 = input("Enter First Child Name: ")
4  child2 = input("Enter Second Child Name: ")
5  name = input("Enter Child Name: ")
6  if name == child1:
7      print("Father:", father)
8      print("Mother:", mother)
9      print("Sibling:", child2)
10 elif name == child2:
11     print("Father:", father)
12     print("Mother:", mother)
13     print("Sibling:", child1)
14 else:
15     print("No information available")
```

Problems Output Debug Console Terminal Ports

```
Enter Father's Name: venkata krishna murtthy
Enter Mother's Name: naga lakshmi
Enter First Child Name: venkat
Enter Second Child Name: nandha
Enter Child Name: venkat
Father: venkata krishna murtthy
Mother: naga lakshmi
Sibling: nandha
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs> |
```

exp14 > ...

```
1 age = int(input("Enter Age: "))
2 bmi = float(input("Enter BMI: "))
3 condition = input("Enter Health Condition (diabetes/bp/normal): ").lower()
4 if condition == "diabetes":
5     print("Diet Plan: Low sugar, whole grains, vegetables.")
6 elif condition == "bp":
7     print("Diet Plan: Low salt, fruits, leafy vegetables.")
8 elif bmi >= 25:
9     print("Diet Plan: Low-fat diet, salads, regular exercise.")
10 elif bmi < 18.5:
11     print("Diet Plan: High-protein diet, milk, eggs, nuts.")
12
13 else:
14     print("Diet Plan: Balanced diet with fruits, vegetables, and plenty of water.")
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs> & "C:\Users\VENKAT SAI.J\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/VENKAT SAI.J/OneDrive/Desktop/cv/bfs/exp14"
```

Enter Age: 25

Enter BMI: 28

Enter Health Condition (diabetes/bp/normal): BP

Diet Plan: Low salt, fruits, leafy vegetables.

PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\cv\bfs> █

exp16 X

exp16 > ...

```
1 class Bird:
2     def can_fly(self):
3         return True
4 class Penguin(Bird):
5     def can_fly(self):
6         return False
7 bird = input("Enter bird name (sparrow/eagle/penguin/parrot): ").lower()
8 if bird == "sparrow":
9     obj = Bird()
10 elif bird == "eagle":
11     obj = Bird()
12 elif bird == "penguin":
13     obj = Penguin()
14 elif bird == "parrot":
15     obj = Bird()
16 else:
17     print("Bird not found!")
18     exit()
19 if obj.can_fly():
20     print(bird.capitalize(), "can fly.")
21 else:
22     print(bird.capitalize(), "cannot fly.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> & "C:\Users\VENKAT SAI.J\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/VENKAT SAI.J/OneDrive/Desktop/AI lab/exp16"

Enter bird name (sparrow/eagle/penguin/parrot): lion

Bird not found!

PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> |

exp17 EXP18 X

EXP18 > classify_fruit

```
1 def classify_fruit(color):
2     color = color.lower()
3     if color == "red":
4         return "Apple"
5     elif color == "yellow":
6         return "Banana"
7     elif color == "orange":
8         return "Orange"
9     elif color == "green":
10        return "Guava"
11    elif color == "purple":
12        return "Grapes"
13    else:
14        return "Unknown Fruit"
15 color = input("Enter the fruit color: ")
16 fruit = classify_fruit(color)
17 print("Fruit:", fruit)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> & "C:\Users\VENKAT SAI.J\AppData\Local\Microsoft\Windows\Python\Python314\python.exe" "c:/Users/VENKAT SAI.J/OneDrive/Desktop/AI lab/EXP18"
Enter the fruit color: green
Fruit: Guava
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> & "C:\Users\VENKAT SAI.J\AppData\Local\Microsoft\Windows\Python\Python314\python.exe" "c:/Users/VENKAT SAI.J/OneDrive/Desktop/AI lab/EXP18"
Enter the fruit color: purple
Fruit: Grapes
PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> 
```

EXP19 > ...

```
1
2  planets = {
3      "Mercury": [1, "Terrestrial"],
4      "Venus": [2, "Terrestrial"],
5      "Earth": [3, "Terrestrial"],
6      "Mars": [4, "Terrestrial"],
7      "Jupiter": [5, "Gas Giant"],
8      "Saturn": [6, "Gas Giant"],
9      "Uranus": [7, "Ice Giant"],
10     "Neptune": [8, "Ice Giant"]
11 }
12 planet = input("Enter planet name: ").capitalize()
13 if planet in planets:
14     position = planets[planet][0]
15     planet_type = planets[planet][1]
16     print(planet, "is the", position, "planet from the Sun")
17     print(planet, "is a", planet_type)
18 else:
19     print("Planet not found")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

thon\Python314\python.exe" "c:/Users/VENKAT SAI.J/OneDrive/Desktop/AI lab/EXP19"

Enter planet name: earth

Planet not found

PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> & "C:\Users\VENKAT SAI.J\AppData\Local

thon\Python314\python.exe" "c:/Users/VENKAT SAI.J/OneDrive/Desktop/AI lab/EXP19"

Enter planet name: earth

Earth is the 3 planet from the Sun

Earth is a Terrestrial

PS C:\Users\VENKAT SAI.J\OneDrive\Desktop\AI lab> █